

AI Native Node OS

AI Native Cluster / Agent Cell Runtime 技术方案

03 Runtime Architecture 基线建议

目标：把 AI Native System 从“应用平台”提升为可安装、可集群、可调度、可隔离执行 Agent Workload 的 AI Native Runtime 基础设施。

【设计决策】一句话架构

AI Native System 将企业 Work 转化为 Run，由 Control Plane 选择 Agent 并将 Agent Cell 调度到 Cluster 中的 Node 执行；Agent Cell 提供隔离的 Linux-like 执行环境，而 Context、Memory、Artifact、State 与 Audit 由平台统一管理。

WORK PLANE	CONTROL PLANE	CLUSTER PLANE	RUNTIME PLANE
Work / Task	Controller / Scheduler	Node / Worker Pool	Agent Cell / Run

版本：v0.1 评审稿

日期：2026-09-26

定位：产品架构 + 技术架构 + Runtime 设计基线

0. 结论先行

本方案建议正式建立 AI Native Node OS，并以 Control Plane + Worker Node 作为集群基本形态。Node 不是一个普通 Linux 上随意安装的应用，而是面向 Agent Workload 的专用计算节点；同时保留 Linux Package 形态，满足开发、POC 与现有数据中心的渐进接入。

架构决策	结论	原因
发布形态	生产标准：AI Native Node OS；兼容：Linux Package / VM Image	兼顾可控性与落地门槛
集群模型	Control Plane + Peer Worker Nodes	避免 Node 互相管理，统一调度与状态
执行单元	Agent Cell 是一级 Runtime 对象	VM / Sandbox / Container 只是实现
隔离策略	RuntimeClass：native / sandbox / microVM / gpu / physical	按风险与资源选择，不强制全部 VM
状态策略	Agent Cell 尽量 Ephemeral，状态外置	Cell 可死、Work 不死
HA	控制面奇数仲裁；双重 Worker + 轻量 Witness	避免双节点脑裂
核心主链	Project → Work → Agent → Run → Agent Cell → Worker → Node → Cluster	产品对象与基础设施对象闭环
Kubernetes 关系	可作为 Execution Provider / 兼容层，而非核心对象模型	避免被 Pod 模型限制

【设计决策】建议作为基线冻结

建议将本方案作为 AI Native System 的“03 Runtime Architecture”主基线；后续产品能力、场景能力包、Harness、Sandbox、Context、Memory、Capability Market 均映射到该执行链。

阅读地图

章节	回答的问题
1-3	为什么要做 Node OS；系统边界是什么；总体四层架构是什么
4-6	核心对象如何连接；Node 内部是什么；两个 Node 如何组成 Cluster
7-10	Control Plane、Worker、Agent Cell、调度与 Reconcile 怎么运行
11-15	状态、网络、存储、安全、HA、观测和成本怎么处理
16-20	怎么发布、怎么安装、接口怎么定义、技术怎么选、PoC 怎么做
21-23	Architecture Decisions、风险/待验证项、官方技术依据

1. 背景与目标

1.1 当前问题

- 如果 AI Native System 只以普通 SaaS / Web 应用方式交付，底层 Agent 的执行环境、资源隔离、生命周期、网络、凭证、审计会散落到各业务模块，无法形成统一 Runtime。
- 如果每个 Research / Coding / Data Agent 服务都自行创建容器或 VM，会重复建设调度、资源管理、安全、成本与恢复机制，三年后极易失控。

- 企业环境中存在裸机、私有云 VM、公有云、GPU 节点、边缘节点甚至机器人控制器，不能把系统绑死在单一 Kubernetes Pod 模型上。
- Agent 的长期身份、Memory、权限与知识不应绑定在某台 VM 上；否则扩缩容、故障迁移、升级和多节点调度会非常困难。

1.2 建设目标

建立一个面向 Human + Agent + Work 的分布式执行底座，使任意加入集群的 AI Native Node 都能够接受 Control Plane 的 Work 调度，为 Agent 创建隔离执行环境，并把运行结果持续沉淀回 Artifact、State、Event、Memory 与 Knowledge。

目标	期望能力
可发布	ISO / VM Image / Linux Package / Cloud Image
可集群	多 Node 组成统一资源池，支持 Worker Pool
可调度	根据 RuntimeClass、资源、策略、数据位置选择 Node
可隔离	native / gVisor / microVM / GPU / Edge 多执行等级
可恢复	Cell 故障后从 Workspace/Checkpoint/Context 恢复
可治理	统一身份、权限、Credential、Network Policy、Audit、Cost
可演进	未来接 Kubernetes、Physical AI、GPU Pool 而不改产品对象模型

1.3 非目标

- 不重写 Linux Kernel。Linux 继续负责进程、内存、文件系统、网络、硬件驱动与虚拟化基础能力。
- 不在第一阶段复刻 Kubernetes 的全部能力。只实现 Agent Workload 必要的 Cluster / Scheduling / Reconcile / Runtime 能力。
- 不把每个 Agent 永久绑定为一台 VM。Agent 是逻辑身份，Agent Cell 是可创建、可销毁、可恢复的运行环境。
- 不把 Chat 作为 Source of Truth。Work、Run、State、Artifact 才是运行系统的核心事实对象。

2. 核心架构原则

【设计决策】 P1 Work-first Control Plane 调度的是 Work / Run，不是“聊天消息”。
【设计决策】 P2 Agent identity != runtime Agent 的身份、角色、Skill、Memory、权限长期存在；Cell/VM 是运行时实例。
【设计决策】 P3 Cell can die, Work must survive 执行环境可被回收或故障，Work 状态与关键结果必须外置并可恢复。
【设计决策】 P4 Control Plane owns lifecycle 只有 Runtime Control Plane 有权创建、迁移、销毁 Agent Cell。业务服务只能提交 Work。
【设计决策】 P5 Node is a resource provider Node 是 Peer Worker，不互相主从；资源、健康和能力统一注册到 Cluster。
【设计决策】 P6 Runtime-agnostic

Agent Cell 抽象高于 Container / gVisor / microVM；底层技术可替换。
【设计决策】 P7 Policy-by-default 网络、Credential、文件挂载、工具能力、成本、超时均由策略显式授予。
【设计决策】 P8 Declarative + Reconcile 系统维护 Desired State 与 Current State 差异，并持续收敛。
【设计决策】 P9 Thin OS, strong control Node OS 尽量最小化、不可随意安装业务软件，降低漂移和攻击面。
【设计决策】 P10 API-first Node 管理、Runtime、Work、Cell、State 均通过明确 API 交互，避免 SSH 脚本式运维。

3. 总体逻辑架构

整体分为 Work Plane、Control Plane、Cluster Plane、Runtime Plane 四层，下方由 AI Native Node OS 和基础设施承载。四层的关键是：上层描述“要完成什么工作”，下层负责“在哪里、用什么隔离等级、由哪个 Agent 运行”。

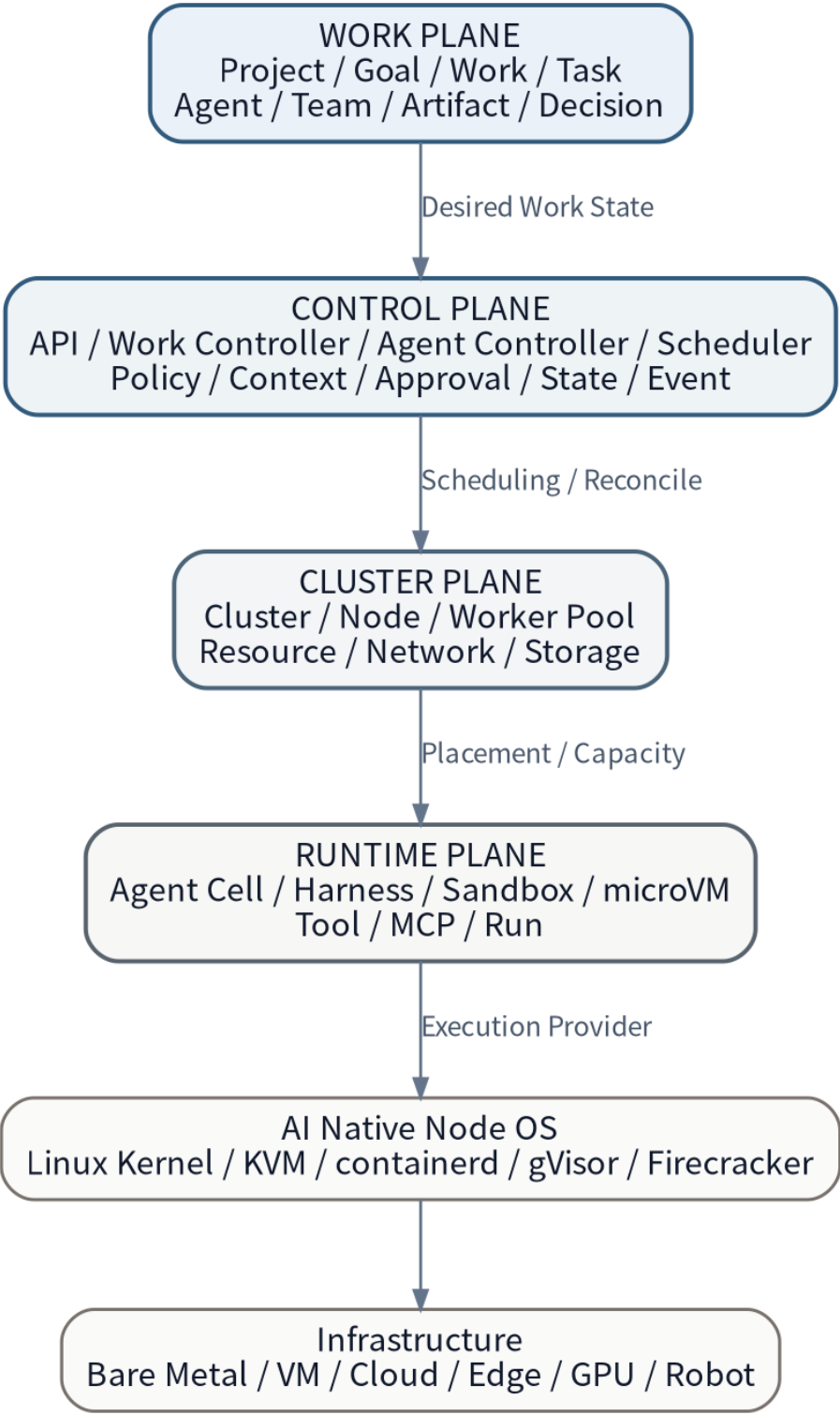


图 3-1 | AI Native System 四层 Runtime 总体架构

层	核心对象	职责
Work Plane	Project / Goal / Work / Task / Agent / Artifact / Decision	定义业务目标、工作对象、执行者与结果
Control Plane	Controller / Scheduler / Policy / Context / State / Event	维护期望状态、调度、治理、恢复
Cluster Plane	Cluster / Node / Worker Pool / Resource / Network / Storage	把多台机器组织成统一资源与能力池

Runtime Plane	Agent Cell / Harness / Sandbox / microVM / Tool / MCP / Run	真正执行 Agent Workload
Node OS	Linux / KVM / containerd / gVisor / Firecracker	提供宿主级隔离、网络、存储与生命周期基础

【设计决策】与 Kubernetes 的边界

借鉴 Kubernetes 的 Desired State、Scheduler、Controller/Reconcile 与 RuntimeClass 思想，但系统的核心对象是 Work / Agent / Run / Agent Cell，而不是 Deployment / Pod / Container。Kubernetes 可以成为一种 Provider，而非产品模型本身。

4. 核心 Object Model

核心对象分成“工作世界”和“运行世界”。二者在 Run ↔ Agent Cell 处连接。这样可以保证产品层不感知具体 VM/容器，同时基础设施层不需要理解复杂业务语义。

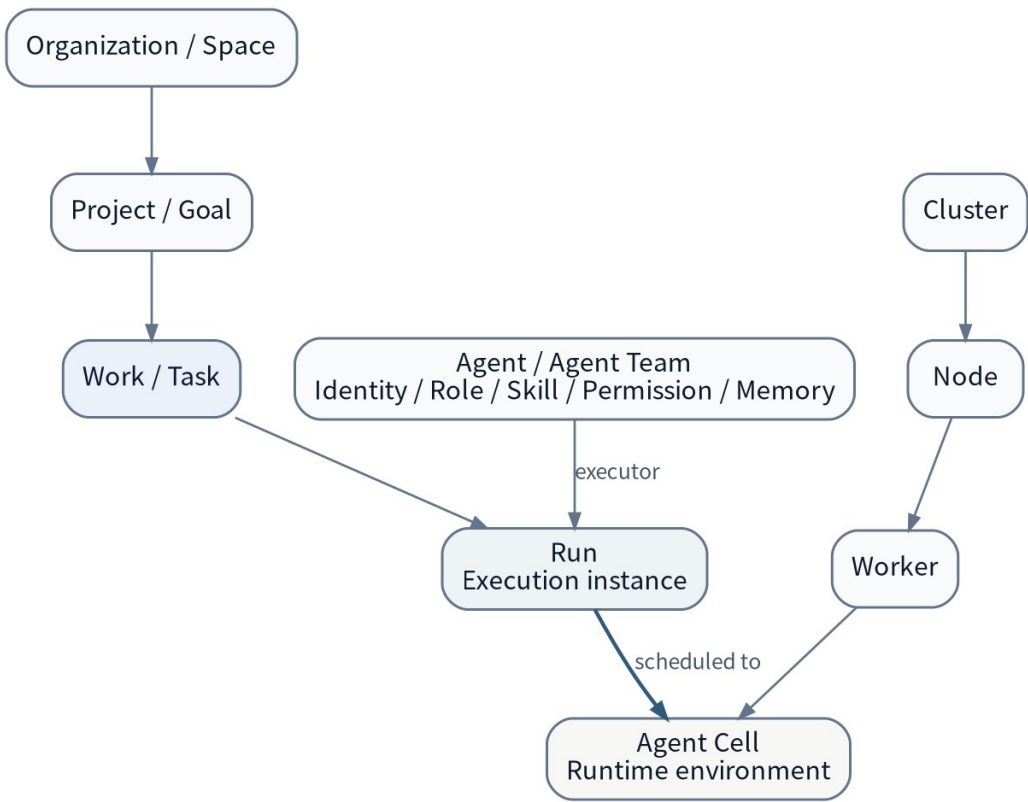


图 4-1 | 业务对象与基础设施对象的连接关系

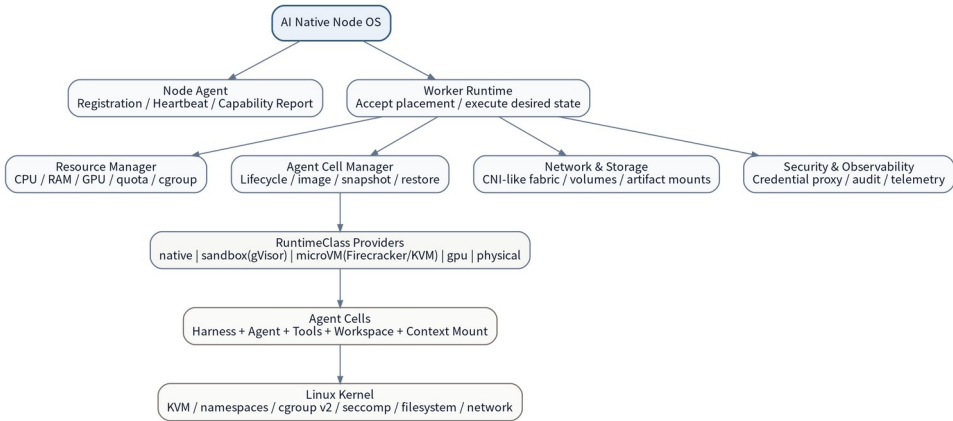
对象	定义	持久性	关键关系
Agent	逻辑执行者：Identity + Role + Model + Skill + Permission + Memory	长期	一个 Agent 可产生多个 Run
Run	一次具体执行实例	长期留痕	属于 Work；被调度到一个 Agent Cell
Agent Cell	Agent 的隔离运行环境	通常短期	承载 Harness、Tools、Workspace
Worker	Node 上的执行守护进程	长期	管理 Cell 生命周期与资源
Node	一台 AI Native OS 机器/VM	长期	向 Cluster 报告资源与能力

Cluster	多个 Node 的逻辑资源池	长期	由 Control Plane 管理
Organization / Space - Project / Goal - Work / Task - Executor: Agent / Agent Team / Workflow / Human - Run - Agent Cell - Worker - Node - Cluster			

5. AI Native Node OS 设计

5.1 Node OS 定位

生产标准形态下，一台安装 AI Native Node OS 的服务器应被视为专用 AI Compute Node，而不是通用 Linux 主机。它的主要使命是接收 Cluster 调度、创建 Agent Cell、提供资源隔离并上报运行状态。



5.2 Node OS 组件

组件	职责	实现建议
Linux Kernel	驱动、进程、内存、网络、KVM、cgroup、namespace	复用成熟 Linux，不改 Kernel
Node Agent	注册、心跳、Capability/Health 上报、配置下发	常驻 daemon，mTLS 接入 Control Plane
Worker Runtime	接受 placement，创建/停止/恢复 Agent Cell	Node 内核心执行服务
Resource Manager	CPU/RAM/GPU/IO 配额与过载保护	cgroup v2 + GPU runtime + quota
Agent Cell Manager	Image、RootFS、snapshot、lifecycle	适配多 RuntimeClass
Network Manager	Cell 网络、Service、egress 策略	CNI-like plugin + eBPF/iptables 可选
Volume Manager	Workspace/Checkpoint/Artifact 挂载	本地缓存 + 共享/对象存储
Credential Agent	Secret 注入、短期 Token、工具授权	Cell 不保存长期密钥
Telemetry Agent	logs/metrics/traces/events/cost	统一上报 Control Plane

5.3 OS 发布策略

阶段	发布方式	建议
MVP	Linux Package + VM Image	先验证 Runtime，不先投入完整 OS 工程

Beta	Minimal Node Image	固定 Kernel/userspace/runtime 版本，减少环境漂移
Production	Immutable AI Native Node OS	只读/不可变根文件系统、签名镜像、A/B 更新、回滚
Enterprise	Bare metal / Private cloud / Cloud image	统一 Cluster Join / Upgrade / Drain / Replace 生命周期

【说明】不要一开始重做 Linux 发行版

第一阶段的价值在 Runtime 和 Control Plane，而不是制作 ISO。先以受控 Linux Image 验证 Node Agent / Worker / Agent Cell；当版本、升级、安全与交付压力出现后再固化为不可变 Node OS。

6. Cluster 与多 Node 架构

多个 AI Native Node 通过 Join Token / Node Certificate 加入同一 Cluster。Control Plane 将 Node 视为 Peer Worker，统一汇聚 CPU、RAM、GPU、Disk、RuntimeClass、Network Zone、Security Level 等能力。

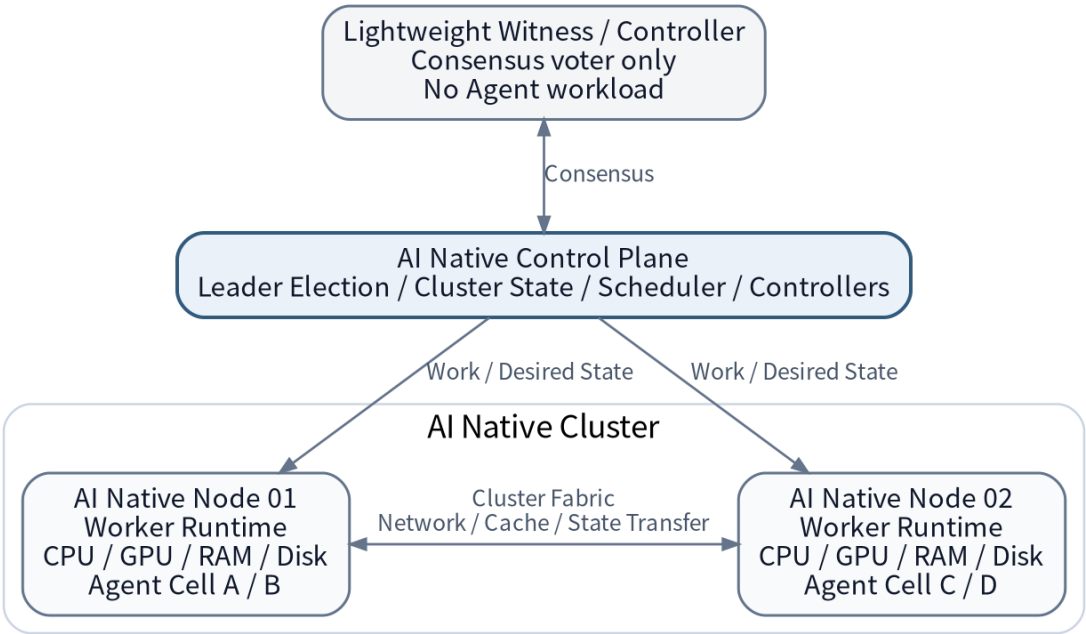


图 6-1 | 双重 Worker + 轻量 Witness 的推荐拓扑

6.1 两 Node 场景

角色	Node 01	Node 02	Witness / Controller
Agent Workload	是	是	否
Worker Runtime	是	是	否
Cluster State voter	可选	可选	是
Leader election	参与	参与	参与
资源规格	高	高	极低
目的	提供算力	提供算力	避免 2 节点脑裂/补足仲裁

【技术依据】为什么需要第三个投票点

如果控制面状态使用 Raft/etcd 一类多数派共识，两节点网络分区时无法既保证一致性又保持双方继续写入。官方 Kubernetes HA 指南同样建议 3 个及以上 control-plane，并指出奇数成员有利于 leader selection；etcd 也建议奇

数成员。第三个节点可以是极轻量 Witness，不承担 Agent 计算。

6.2 Worker Pool

AI Native Cluster

- General Pool -> 普通 CPU Agent
- Coding Pool -> 编译/测试/代码执行
- GPU Pool -> CUDA / Model workload
- Secure Pool -> 高隔离 / 敏感数据
- Browser Pool -> 浏览器自动化
- Physical Pool -> Factory / Robot / Edge runtime

Worker Pool 是调度约束与资源治理边界，不应与组织部门直接绑定。一个 Node 可属于一个主 Pool，并通过 Label/Capability 暴露更多特征。

7. Control Plane 设计

7.1 组件职责

组件	核心职责
API Server / Gateway	统一写入口；认证；版本化 API；幂等请求
Work Controller	维护 Work 生命周期，生成/更新 Run，检查完成条件
Agent Controller	解析 Agent Definition、Team、Skill、Model、Policy
Scheduler	Node/Pool 过滤、评分、Placement、重调度
Run Controller	维护 Run Desired/Current State；失败重试；终止；超时
Context Controller	为每次 Run 组装 Task/Project/Memory/Knowledge/Data/Permission Context
Policy Controller	权限、网络、RuntimeClass、Tool/MCP、Cost、Approval Gate
Node Manager	Node join/heartbeat/drain/cordon/upgrade/capability
State Store	强一致元数据；Cluster/Run/Cell 状态
Event/Audit Store	不可变事件、审计、日志索引、执行轨迹

7.2 Desired State / Reconcile

Control Plane 不通过“脚本执行完就算成功”的方式管理 Worker，而是持续比较 Desired State 与 Current State。例如 Desired State 要求某 Run 为 Running，而 Cell 不存在，则 Reconciler 重新选 Node 并创建 Cell；如果 Work 已取消，则回收 Cell 并收敛为 Cancelled。

Desired: Run#R128 = Running, runtimeClass=microvm, cpu=4, mem=8Gi
Current: AgentCell missing (Node01 lost)

Reconcile:
1. validate policy & quota
2. choose eligible Node02
3. restore workspace/checkpoint
4. create AgentCell#C991
5. attach Context/Credential/Tools
6. resume Harness
7. update Run.currentState = Running

8. Worker Runtime 与 Agent Cell

8.1 Agent Cell 定义

【设计决策】 Agent Cell 是一级对象	
Agent Cell = 一次可调度、可隔离、可观测、可恢复的 Agent 运行环境。它可以由 Container、gVisor Sandbox 或 Firecracker microVM 实现，因此产品层不要把对象命名为 VM。	
Agent Cell 内部	说明
Minimal Linux-like runtime	进程、文件系统、网络、依赖环境
Agent Harness	Loop、Tool use、checkpoint、goal completion、retry
Agent Process	具体模型与 Agent logic
Tool / MCP Runtime	浏览器、Python、Git、Office、企业系统连接器
Workspace	当前 Work 的临时/持久化工作目录
Context Mount	动态装配的 Task/Project/Knowledge/Data/Memory
Credential Proxy	短期 token / scoped secret
Telemetry Sidecar / SDK	Run event、resource、cost、trace

8.2 Agent 与 Cell 解耦

Agent Definition (long-lived)
Identity / Role / Model / Skill / Permission / Memory
instantiate for a Run
v
Agent Cell (ephemeral runtime)
RootFS / Process / Tools / Workspace / Context
v
Run (durable execution record)

同一个 Research Agent 可以今天在 Node01 的 gVisor Cell 中执行，明天在 Node02 的 microVM 中执行；Agent 的长期 Memory、Skill、History 不因此改变。

9. RuntimeClass 与隔离策略

RuntimeClass	实现	适用场景	优点	约束
native	process / runc container	平台内部可信任务	密度与性能最高	隔离最弱
sandbox	gVisor / runsc	默认第三方代码、浏览器、Coding Agent	比普通容器更强隔离；兼容 OCI/containerd	存在额外性能开销
microvm	Firecracker / KVM	多租户、高风险代码、强安全边界	硬件虚拟化隔离；独立 guest kernel	依赖 KVM；VM 宿主可能需要 nested virtualization
gpu	native/sandbox + GPU runtime	模型、CUDA、视觉/训练	支持 GPU 调度	GPU 隔离和驱动复杂
physical	edge/robot provider	机器人、工厂、设备控制	靠近设备与实时数据	需处理断网、实时性、设备安全

【已验证/官方能力】 gVisor 作为默认候选
gVisor 是用户态 application kernel，可通过 runsc 与 containerd/Kubernetes 集成；其生产文档明确面向不可信代码和多租户隔离。2026-09-17 官方还宣布支持 systemd，使其更接近“Linux-like Agent Cell”的使用体验。
【已验证/官方能力】 Firecracker 作为强隔离候选

Firecracker 使用 Linux KVM 创建 microVM，面向安全、多租户、低开销执行。官方设计要求 Linux host + Linux guest；生产环境建议配合 jailer、seccomp、cgroup 和 namespace。

10. Work → Run → Agent Cell 调度流程

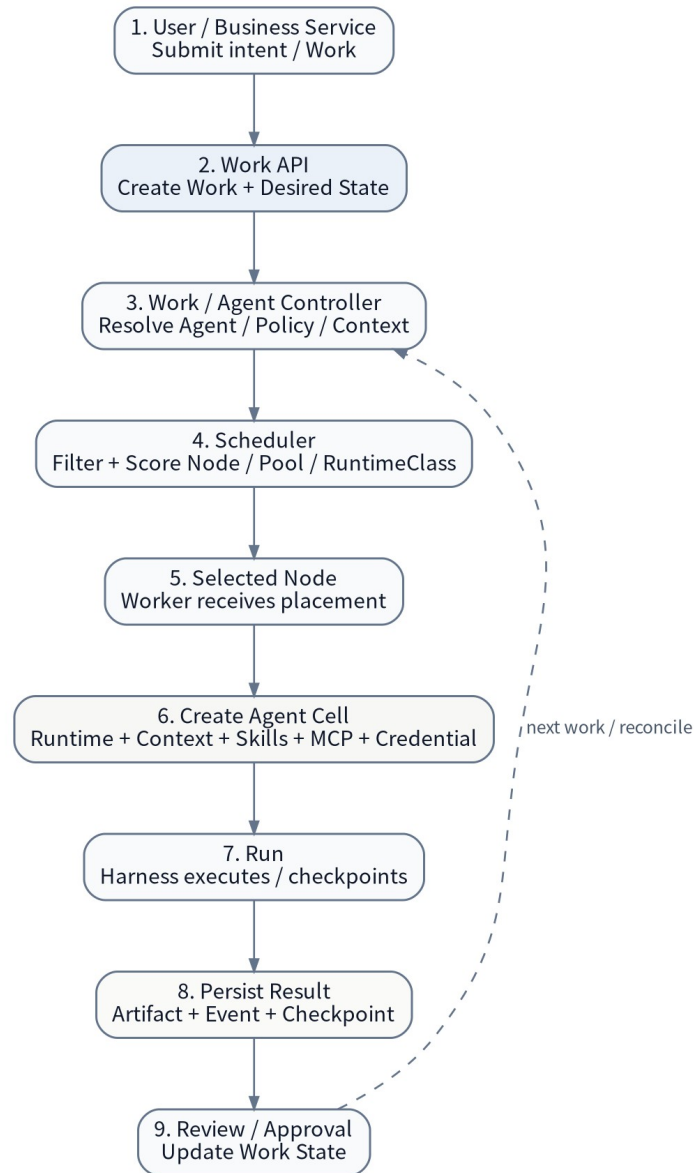


图 10-1 | 从 Work 到 Agent Cell 的标准执行链

10.1 Scheduler Filter

- Pool / Zone：是否满足 General / GPU / Secure / Physical 等池约束。
- RuntimeClass：Node 是否支持 gVisor / KVM / GPU / Edge provider。
- Resource：CPU、RAM、GPU、Ephemeral Disk、IOPS、Network bandwidth。
- Policy：数据地域、安全等级、租户隔离、Credential、Network egress。
- Data affinity：Context / Dataset / Model 是否在本地或可快速挂载。
- Health：Node 是否 Ready、是否 Drain、是否资源压力或版本不一致。

10.2 Scheduler Score

评分项	目的
Resource balance	避免局部过载，保持资源利用率
Image/cache locality	优先已有 RootFS / Tool image 的 Node
Data locality	降低大型数据/模型搬运
Cost	优先更低成本的 CPU/GPU/Cloud pool
Reliability	避开近期频繁失败节点
Latency	Physical/interactive Agent 靠近用户或设备

10.3 Reconcile 与恢复

Scheduler 负责“放在哪里”，Controller 负责“系统最终应该处于什么状态”。节点掉线后不直接把原 VM 当作真相，而是以 Run State + Checkpoint + Workspace + Event Log 为恢复依据。

11. Context、Memory、Artifact 与状态外置

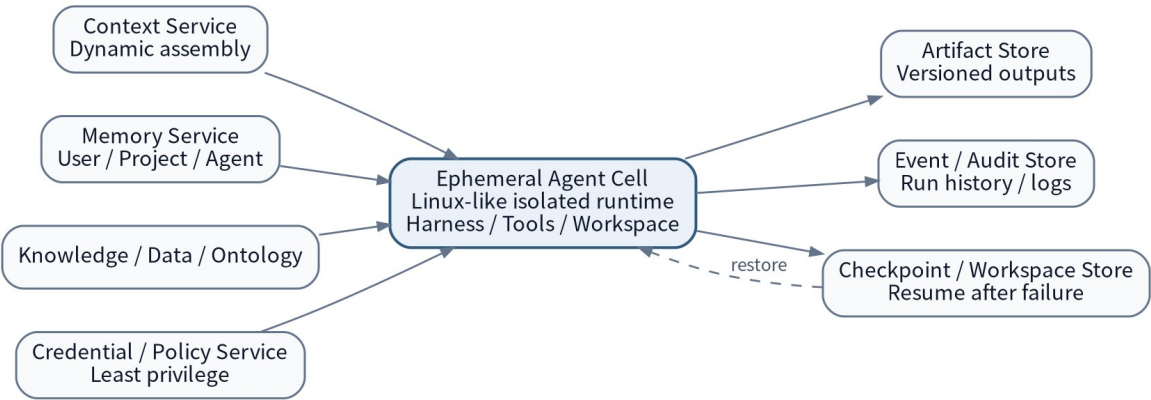


图 11-1 | Agent Cell 的状态外置与恢复模型

数据/状态	归属	是否进入 Cell	持久化建议
Run State	Control Plane	只读/局部缓存	强一致 State Store
Context	Context Service	动态 Mount / Inject	来源可追溯、按 Run 装配
Session/Working Memory	Run/Agent	可注入	Checkpoint + Memory Service
User/Project/Agent Memory	Memory Service	按权限读取	Cell 外持久化
Knowledge/Data/Ontology	Enterprise Brain / Data Asset	引用/查询/挂载	Cell 外 Source of Truth
Artifact	Artifact Store	输出后上传	版本化、Owner、Source、Relation
Workspace	Run	可本地缓存	重要目录增量同步/快照
Credential	Credential Service	短期 token	不写入长期镜像/磁盘
Logs/Events	Telemetry / Event Store	产生	集中收集，不依赖本地

【设计决策】关键原则：Cell 可死，Work 不死

任何导致 Run 无法在另一 Node 恢复的本地状态，都应被明确标记并尽量外置；本地缓存可以提升性能，但不能成为唯一事实来源。

12. 网络与存储架构

12.1 Network Fabric

网络域	用途	策略
Control Network	Control Plane ↔ Node Agent/Worker	mTLS、固定 API、禁止 Agent 直接访问
Cluster Data Network	Node ↔ Node / Storage / Cache	分区、限流、可加密
Agent Cell Network	Agent 访问 Tool/MCP/Internet/API	默认 deny + allowlist / policy
Service Network	平台内部服务发现	服务身份 + mTLS
Physical/Edge Network	设备/机器人/工厂协议	独立安全域、最小权限、故障隔离

12.2 Storage Layers

层	用途	建议
Node Local Cache	Image / RootFS / Model / package cache	可丢失，按容量回收
Workspace Store	Run 工作目录、checkpoint	支持快照/增量同步
Artifact Store	文档、代码、报表、图片、业务输出	对象存储 + 元数据 + 版本
State Store	Cluster/Run/Cell 元数据	强一致，小数据
Event/Log Store	运行事件、审计、日志、trace	高吞吐 append/index
Knowledge/Data Store	企业知识、数据资产、本体	独立服务，不塞进 Runtime DB

13. 安全、权限与 Credential

13.1 信任边界

边界	默认信任	核心控制
Control Plane	高	管理面隔离、MFA/RBAC、审计
Node OS	高但可被攻破	Secure boot/签名镜像/最小化软件/快速升级
Worker Runtime	高	专用 service identity、最小 host privilege
Agent Cell	低/不可信	sandbox/microVM、resource/network/file policy
Tool/MCP	按能力授权	Capability token、审批、参数校验、审计
External Internet/API	不可信	egress allowlist、proxy、DLP/Content policy

13.2 Credential 模型

Agent 不应获得企业系统的长期账号密码。推荐 Credential Broker 根据 Run、Agent、Tool、Task、Approval 动态签发短期、可撤销、最小权限的 Credential；Cell 销毁后凭证自然失效。

CredentialRequest
subject: Agent#research-01
run: Run#R128
capability: notion.write
scope: Project/AI-Native-System
ttl: 30m
approval: required-if-external-publish

13.3 Agent Cell 安全基线

- 默认无 root host capability；禁止直接访问 host socket、/dev/kvm、containerd socket。
- 网络默认 deny；按 Tool/MCP/URL/Service allowlist 放行。
- Workspace 与 Context 只挂载必要范围；跨项目数据必须重新授权。
- 所有 Tool 调用记录 Agent、Run、输入参数摘要、结果状态与外部副作用。
- 高风险写操作可插入 Human Approval Gate。

14. 高可用与故障恢复

14.1 故障模型

故障	检测	系统动作
Agent process crash	Harness heartbeat / exit code	Cell 内重启；达到阈值后 Run retry
Agent Cell crash	Worker status missing	同 Node 重建或重新 placement
Worker crash	Node Agent / Control heartbeat	标记 Node NotReady；Run 重调度
Node crash	heartbeat timeout	Node fencing；从 checkpoint 在其他 Node 恢复
Control Plane instance crash	leader election / load balancer	由其他实例继续服务
Network partition	lease/heartbeat/quorum	多数派继续；少数派禁止写入关键状态
Storage outage	I/O error / health	暂停需要持久化的 Run；防止错误完成

14.2 两 Node + Witness

对于只有两台高规格服务器的客户，不建议为了控制面 HA 再购买第三台同规格服务器。可以部署一个极轻量 Witness/Controller 作为共识投票成员，只保存 Cluster State/参与 Leader Election，不承载 Agent Cell。

【技术依据】 技术依据

Kubernetes 官方 HA 文档要求 3 台及以上 control-plane，并指出奇数成员有利于 leader selection；其 etcd 运维文档明确建议 odd number of members。

14.3 Node Drain / Upgrade

Cordon Node -> stop new placement
Drain Node -> checkpoint / finish / reschedule existing Runs
Upgrade OS -> atomic image update / reboot
Health Gate -> validate Node version/runtime
Uncordon -> rejoin scheduling pool

15. Observability、Audit 与 Cost

维度	最小数据模型	用途
Run Trace	WorkId / RunId / AgentId / CellId / NodeId / span	端到端追踪
Events	state transition + timestamp + actor + reason	重放、排障、审计
Logs	stdout/stderr/harness/tool/runtime	技术排障
Metrics	CPU/RAM/GPU/IO/Network/queue/startup/failure	容量与 SLO

Model Cost	model/token/request/cache	模型成本
Tool Cost	browser/API/MCP/external service	外部服务成本
Infra Cost	Node time/GPU time/storage/egress	基础设施成本
Audit	permission/credential/tool/write/ approval	治理与合规

【设计决策】成本对象必须落到 Work/Run

不能只统计“模型 Token”。真正的 AI Native 成本应可回溯到 Project → Work → Run → Agent → Model/Tool/Node，才能支持项目核算、预算、配额与优化。

16. 安装、发布与部署形态

形态	用户动作	适用场景	备注
Standalone	安装后 Control Plane + Worker 同机	个人/开发/POC	一台即可运行
Linux Package	在现有 Linux 安装 worker/controller package	存量数据中心	兼容性最高、环境漂移较大
VM Image	导入 VMware/KVM/私有云模板并 Join Cluster	企业首选早期形态	便于统一版本
AI Native Node OS ISO	裸机/VM 安装专用 OS	成熟生产标准	最可控、升级一致
Cloud Image	云市场/镜像库创建节点	公有云/弹性扩容	支持自动注册和 autoscaling

16.1 Join Cluster 流程

1. Boot / install AI Native Node OS
2. Enter Control Plane endpoint + one-time Join Token
3. Establish node identity (certificate)
4. Inventory hardware: CPU/RAM/GPU/Disk/KVM/RuntimeClass
5. Download cluster policy & runtime images
6. Health check
7. Node.status = Ready
8. Scheduler starts placement

16.2 Node 生命周期

生命周期至少包含 Provisioning → Joining → Ready → Pressure/Degraded → Cordoned → Draining → Upgrading → Ready，以及 Lost/Retired。不要仅用“在线/离线”两个状态。

17. API 与声明式对象草案

17.1 Node

```
apiVersion: runtime.ainative.io/v1alpha1
kind: Node
metadata:
  name: node-01
  labels:
    pool: general
    zone: dc-a
spec:
  roles: [worker]
status:
  capacity:
```

```
cpu: "64"
memory: "256Gi"
gpu.nvidia: "2"
runtimeClasses: [native, sandbox, microvm, gpu]
virtualization:
  kvm: true
conditions:
  ready: true
```

17.2 AgentCell

```
apiVersion: runtime.ainative.io/v1alpha1
kind: AgentCell
metadata:
  name: cell-r128-01
spec:
  runRef: R128
  agentRef: research-agent
  runtimeClass: sandbox
  resources:
    cpu: "4"
    memory: "8Gi"
  capabilities: [browser, python, github, notion]
  contextRef: ctx-r128
  networkPolicy: restricted
  lifecycle:
    mode: ephemeral
  checkpoint:
    enabled: true
status:
  phase: Running
  node: node-02
```

17.3 Run

```
kind: Run
spec:
  workRef: W203
  executor:
    type: Agent
    ref: research-agent
  goal: "完成目标产品技术研究并形成报告"
  completionPolicy: goal-judge-v1
  approvalPolicy: publish-external
  budget:
    modelCost: 30USD
    wallTime: 2h
status:
  desiredState: Running
  currentState: Running
  agentCellRef: cell-r128-01
```

【说明】API 设计原则

这些 YAML 不是要求复制 Kubernetes CRD，而是借用声明式表达方式。最终 API 可以是 protobuf/REST/SQL-backed object；关键是对象字段和状态机必须稳定。

18. 技术选型建议

领域	MVP 建议	中长期选择	判断
Host OS	受控 Linux LTS image	Immutable Node OS	先 Runtime，后 OS 产品化
Container runtime	containerd	containerd	成熟、生态兼容
Sandbox	gVisor runsc	gVisor / Kata 可评估	默认不可信 workload
microVM	Firecracker + KVM	Firecracker / Cloud Hypervisor 可评估	高隔离池
State store	PostgreSQL + lease/lock 或 embedded consensus prototype	etcd/consensus store + relational metadata	分离强一致状态与业务元数据
Event	Postgres append / NATS JetStream	Kafka/Pulsar/NATS 按规模	先简后繁
Artifact	S3-compatible object store	对象存储 + content addressable cache	避免塞 DB
Network	Linux bridge/CNI-like plugin	eBPF/Cilium-like data plane 可评估	先满足 policy + service discovery
Observability	OpenTelemetry	OpenTelemetry + metrics/log backend	RunId 贯穿
Identity	mTLS node identity + OIDC users	SPIFFE/SPIRE 类 workload identity 可评估	不要共享长期 secret

18.1 为什么不是直接 Kubernetes

第一版可以复用 Kubernetes 或其组件做快速验证，但产品核心模型不应直接等于 Pod/Deployment。原因是 Agent 需要长期 Identity/Memory、Work-aware recovery、Context Assembly、Tool/MCP/Credential、Approval、Artifact/Decision 等语义。将这些作为 Sidecar 注入 Kubernetes 会很快变成“在 K8s 上堆平台”；我们更需要一个以 Agent Work 为中心的控制面。

18.2 为什么需要多 RuntimeClass

不同企业环境和 Agent 风险等级差异很大。gVisor 官方提供 containerd runtime handler/RuntimeClass 集成，并强调其多租户隔离用途；Firecracker 依赖 KVM，若 Node 本身运行在 VM 中则需要底层支持 nested virtualization。因此 Runtime 必须是可插拔 Provider。

19. 分阶段实施路线

阶段	目标	核心交付	暂不做
Phase 0 架构验证	证明 Work→Run→Cell→Node 链闭环	单机 Control+Worker；AgentCell API；container/gVisor；状态外置	自研 OS、HA、跨 Node
Phase 1 2 Node Cluster	证明多节点调度/恢复	Node join/heartbeat；Scheduler；Worker Pool；Node lost reschedule	复杂 autoscaling
Phase 2 安全与 HA	达到企业可用	3 voter control plane；microVM；Credential/Network Policy；Drain/Upgrade	全球多 Region
Phase 3 Node OS 产品化	形成标准交付	Immutable image；签名/升级/rollback；VM/ISO/Cloud image	大规模边缘
Phase 4 AI Native Fabric	扩展异构执行	GPU/Physical/Edge provider；capacity/cost scheduling	一次性做全场景

19.1 第一阶段最小闭环

Create Work
-> Create Run
-> Resolve Agent + Context + Policy
-> Scheduler selects Node
-> Worker creates Agent Cell
-> Harness executes
-> Tool/MCP calls audited
-> Artifact uploaded
-> Run completed
-> Work state updated
-> Cell destroyed

【设计决策】第一阶段不要追求“像 Kubernetes 一样完整”

只要把 Work-aware scheduling、AgentCell lifecycle、state externalization、failure recovery 四个差异化能力跑通，就证明底层路线成立。

20. PoC 方案与验收

20.1 建议 PoC 环境

资源	建议配置
Node 01	8-16 vCPU / 32GB RAM；支持 gVisor；若验证 Firecracker 则需要 KVM
Node 02	8-16 vCPU / 32GB RAM；同上
Witness/Controller	2 vCPU / 4GB RAM 即可，主要用于控制状态与仲裁验证
Shared storage	S3-compatible object store + PostgreSQL
Agent workloads	Research Agent + Coding Agent 两类即可

20.2 必测场景

1. 创建 Work 后自动生成 Run，并在两个 Node 中选择满足资源/RuntimeClass 的节点。

2. 同一 Agent 连续执行两个 Run，分别调度到不同 Node，Memory/Identity 不丢失。

3. 运行中强制杀掉 Agent Cell，Run 能检测失败并自动重建。

4. 运行中断掉 Node01，未完成 Run 可在 Node02 从 checkpoint/workspace 继续或安全重试。

5. 限制 Agent Cell 访问某外部域名/API，验证 Network Policy 生效。

6. 给 Coding Agent 使用 sandbox；给高风险脚本使用 microVM（若 KVM 环境可用）。

7. Run 完成后 Cell 销毁，Artifact、Event、Audit、Cost 仍可查询。

8. 执行 Node Drain，将新任务停止调度到该 Node，并迁移/结束现有 Run。

20.3 验收维度

维度	验收问题
正确性	Work / Run / Cell / Node 状态是否一致；失败后是否能收敛
隔离	不同 Agent 是否无法越权访问 host/其他 Cell/其他项目 Context
恢复	Cell/Node 故障后，Artifact/Run/Checkpoint 是否可恢复
调度	RuntimeClass/资源/Pool/Policy 是否影响 Placement
观测	能否从 Work 一路追踪到 Model/Tool/Cell/Node

运维	Node join/drain/upgrade/replace 是否无需手工 SSH 改状态
----	--

21. Architecture Decision Records (ADR)

ADR	决策
ADR-01	生产发布 AI Native Node OS；保留 Linux Package / VM Image 兼容形态。
ADR-02	多 Node 组成 AI Native Cluster，Node 之间为 Peer Worker。
ADR-03	使用 Control Plane / Worker 模式，禁止业务服务直接创建 Cell。
ADR-04	Agent 与计算环境解耦；Agent Definition 长期存在，Cell 短期存在。
ADR-05	Agent Cell 作为 Runtime 一级对象，VM/Container/Sandbox 为 Provider。
ADR-06	Agent Cell 默认 Ephemeral；State/Memory/Artifact/Credential 外置。
ADR-07	支持 native / sandbox / microVM / gpu / physical 多 RuntimeClass。
ADR-08	Control Plane 采用 Desired State + Reconcile，Run 为核心执行记录。
ADR-09	生产 HA 使用奇数投票成员；2 Worker 场景加轻量 Witness。
ADR-10	Node 按 Capability/Label 进入 Worker Pool；Scheduler 统一 Placement。
ADR-11	Kubernetes 可集成但不是 Source of Truth；核心对象不以 Pod 为中心。
ADR-12	Work → Run → Agent Cell → Worker → Node → Cluster 作为 Runtime 主链。
ADR-13	Context 与 Memory 分离；Context 每次 Run 动态装配。
ADR-14	Artifact 是一等对象，不能只保留聊天 Answer。
ADR-15	Credential 使用短期、可撤销、最小权限授权，避免 Agent 持有长期密钥。

22. 风险、待验证项与下一轮设计问题

优先级	问题	当前建议
P0	Agent Cell 是 “一 Agent 一 Cell” 还是 “一 Run 一 Cell” ？	默认一 Run 一 Cell；长驻 Agent 可作为特殊 lifecycle
P0	Checkpoint 语义做到进程快照还是业务级 checkpoint？	先业务级：Workspace + Harness state + event cursor；不要先做 VM memory snapshot
P0	Control Plane State Store 用 PostgreSQL 还是 etcd？	PoC 可 PG；进入 HA 前做一致性/故障模型基准后再定
P0	Node OS 是否必须 Immutable？	MVP 否；生产标准是
P1	gVisor 对浏览器、编译、Office、Docker-in-sandbox 兼容性	按场景建立兼容矩阵与 benchmark
P1	Firecracker 在客户 VM 中的 nested virtualization 可用性	Node inventory 自动探测 KVM；不可用则不暴露 microVM RuntimeClass
P1	GPU workload 的强隔离	单独设计 GPU RuntimeClass，不强行套普通 microVM
P1	Agent Cell 网络与 MCP 权限的统一策略模型	Policy Engine 使用 Capability + Resource + Action + Scope
P2	是否构建自有 CNI/CSI	前期复用，规模与差异化需求明确后再决定

P2	是否支持跨 Cluster Work 迁移	先单 Cluster; 后续由 Global Control Plane 解决
----	-----------------------	---

【设计决策】 下一步最重要的不是继续扩模块

下一轮应该先把三个规格写清楚：① Run 状态机；② Agent Cell 状态机与 API；③ Node/Worker/Scheduler 协议。它们决定 Runtime 是否真正可实现。

23. 官方技术依据与参考

来源	用于本方案的事实依据	链接
Kubernetes High Availability	Creating Highly Available Clusters with kubeadm: 3 个及以上 control-plane; 奇数成员有利于 leader selection。	https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability/
Kubernetes / etcd	Operating etcd clusters for Kubernetes: etcd 是 leader-based distributed system, 建议 odd number of members。	https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/
Firecracker Design	Firecracker 使用 KVM 创建 microVM, 面向多租户隔离; 生产建议 jailer、seccomp、cgroups、namespaces。	https://github.com/firecracker-microvm/firecracker/blob/main/docs/design.md
Firecracker Getting Started	Firecracker 需要 Linux KVM /dev/kvm; VM 环境可能需要 nested virtualization。	https://github.com/firecracker-microvm/firecracker/blob/main/docs/getting-started.md
gVisor Overview	gVisor 是 userspace application kernel, 提供 Linux-like interface; runsc 可集成 Docker/Kubernetes。	https://gvisor.dev/docs/
gVisor Production	gVisor 面向不可信代码、多租户和敏感 workload 的额外隔离层, 并提示存在性能开销。	https://gvisor.dev/docs/user_guide/production/
gVisor + containerd	gVisor 提供 containerd shim 与 RuntimeClass 集成方式。	https://gvisor.dev/docs/user_guide/containerd/quick_start/
gVisor systemd 2026-09-17	gVisor 官方新增 systemd 支持, 使 sandbox 更接近完整 Linux userspace。	https://gvisor.dev/blog/2026/09/17/systemd-in-gvisor/
AWS Bottlerocket	Bottlerocket 是面向容器 workload 的 Linux-based OS, 只包含最小必要软件, 是 Node OS 产品化的参考。	https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-bottlerocket.html

【技术依据】 证据边界

上述资料用于验证底层技术可行性与业界架构事实；本方案中的 AI Native Object Model、Agent Cell、Work-aware Scheduler、Context/Memory/Artifact 关系、发布路线与 ADR 均为本项目的设计判断，尚需通过 PoC 与工程验证。

附录 A | 建议的 Runtime 核心状态机

A.1 Run State

```
Pending -> Scheduled -> Starting -> Running -> WaitingApproval -> Succeeded
      |           |           |
      |           |           |-> Failed
      |           |-> Paused
      |-> Unscheduleable
Any non-terminal -> Cancelling -> Cancelled
```

A.2 Agent Cell State

```
Requested -> Pulling/Preparing -> Creating -> Ready -> Running
                                     |
                                     |-> Checkpointing
```

```
NodeLost -> Unknown -> Recreate / Abandon
```

A.3 Node State

```
Provisioning -> Joining -> Ready -> Pressure/Degraded
                        |
                        |-> Cordoned -> Draining -> Upgrading -> Ready
                        |-> Rejected
Ready/Degraded -> Lost -> Retired / Rejoin
```

附录 B | 术语建议

不要用	建议用	原因
Master / Slave	Control Plane / Worker Node	语义更准确，也符合现代集群术语
Agent VM	Agent Cell	避免把产品对象锁死在 VM 实现
Pod	Agent Cell / Runtime Unit	保留 Agent 工作语义
知识库	Knowledge / Data Asset / Ontology / Memory 分开	不同生命周期和职责
Agent 进程	Agent Definition + Run + Cell	区分身份、执行记录和计算环境